

Conferencia 2: el oficio algorítmico

Diseño y Análisis de Algoritmos · Ciencia de la Computación · MatCom · 2026

La conferencia anterior fijó cómo se mide un algoritmo. Esta fija cómo se trabaja. En vez de enunciar el método y pasar a otra cosa, lo vamos a recorrer entero, una vez, sobre un solo problema. Al terminar vas a haber visto las cinco etapas completas, y vas a saber por qué la última —demostrar que no se puede hacer mejor— nunca es una afirmación sobre el problema a secas.

1. Las cinco preguntas

Ante un problema computacional, el trabajo consiste en responder cinco preguntas, en este orden:

1. **¿Cuál es el problema?** Enunciarlo con precisión: qué es la entrada, qué es la salida, qué pasa en los casos raros.
2. **¿Qué algoritmo lo resuelve?** Proponer un procedimiento.
3. **¿Es correcto?** Demostrar que para toda entrada válida devuelve la salida correcta.
4. **¿Cuánto cuesta?** Analizar tiempo y memoria en el modelo de la conferencia 1.
5. **¿Se puede hacer mejor?** Buscar una cota mínima: un costo por debajo del cual ningún algoritmo puede resolver el problema.

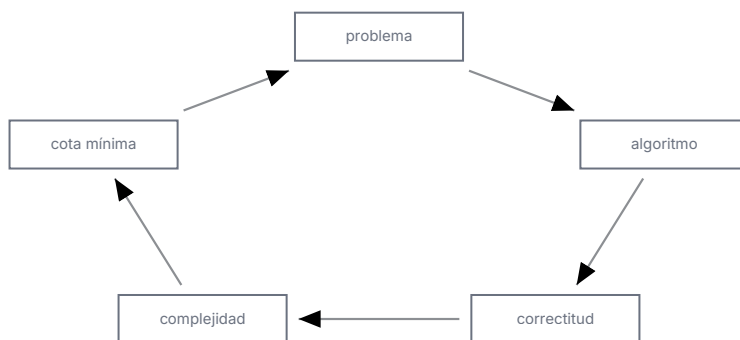


Figura 1. El ciclo que recorre cada conferencia del curso. Hoy lo damos entero sobre un solo problema.

Ninguna se puede saltar. Un algoritmo sin la tercera es una conjetura: funciona en los casos que probaste. Un análisis sin la quinta no sabe si vale la pena seguir buscando, y la mayor parte del esfuerzo desperdiciado en algoritmos se gasta optimizando cosas que ya eran óptimas, o abandonando búsquedas que tenían margen.

Hoy recorreremos el ciclo completo sobre el problema del par de puntos más cercanos. De la conferencia 6 en adelante el ciclo queda implícito y las clases se organizan por técnica, pero las cinco preguntas siguen ahí, y son las que se evalúan.

2. El problema

Par más cercano. Dado un conjunto P de n puntos en el plano, encontrar el par de puntos distintos $p, q \in P$ que minimiza la distancia euclidiana

$$d(p, q) = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}.$$

Tres precisiones que un enunciado tiene que dar, y que casi nunca se dan:

- **Empates.** Si varios pares alcanzan el mínimo, devolvemos cualquiera. Si nos interesaran todos, sería otro problema y podría haber $\Theta(n^2)$ pares empatados: basta poner los n puntos en los vértices de un polígono regular.
- **Puntos repetidos.** Si P es un conjunto, no los hay. Si es una lista y admite repeticiones, la respuesta es 0 y el problema se vuelve el de decidir si hay repetidos, que es exactamente de lo que trata la sección 8.
- **La raíz cuadrada.** $d(p, q) < d(r, s)$ si y solo si $d(p, q)^2 < d(r, s)^2$, así que todos los algoritmos comparan distancias al cuadrado y sacan la raíz una sola vez, al final. Ahorra n^2 raíces y evita el error de redondeo que introducen.

Conviene separarlo de un problema parecido y más caro: **para cada punto, hallar su vecino más cercano.** Ahí hay n respuestas en lugar de una. El par más cercano es el mínimo de esas n respuestas, así que resolver el segundo resuelve el primero, pero no al revés. Distinguir dos problemas que suenan igual es parte del oficio, y es la primera cosa que se hace mal.

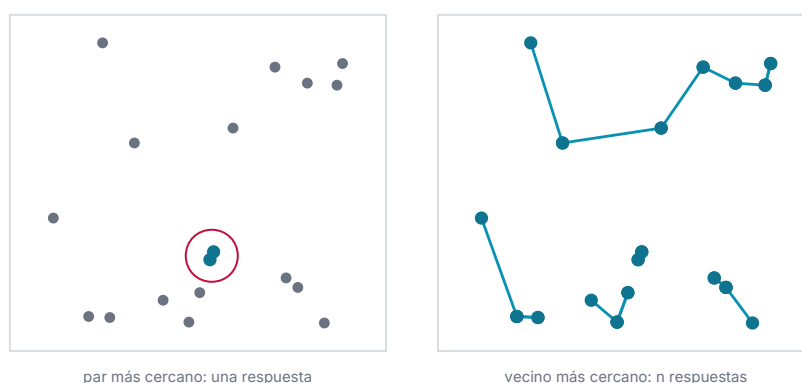


Figura 2. El mismo conjunto, dos preguntas distintas. A la izquierda el par más cercano, una sola respuesta, marcada en rojo porque a esta escala los dos puntos casi se tocan. A la derecha el vecino más cercano de cada punto.

3. Calentamiento: una dimensión

Antes del plano, la recta. Dados n números reales, ¿cuáles son los dos más cercanos? La idea es inmediata: ordenar y mirar pares consecutivos.

Lema. Si $x_1 \leq x_2 \leq \dots \leq x_n$, el par más cercano es consecutivo.

Demostración. Sea (x_i, x_j) con $i < j$ el par más cercano, y supongamos $j > i + 1$. Entonces $i < i + 1 < j$, y como la sucesión está ordenada, $x_j - x_{i+1} \leq x_j - x_i$. El par (x_{i+1}, x_j) es entonces al menos tan cercano, y es distinto de (x_i, x_j) . Repitiendo el argumento llegamos a un par consecutivo que sigue siendo mínimo. $\square$

Con el lema, el algoritmo es ordenar en $O(n \log n)$ y recorrer los $n - 1$ pares consecutivos en $O(n)$. Total $O(n \log n)$.

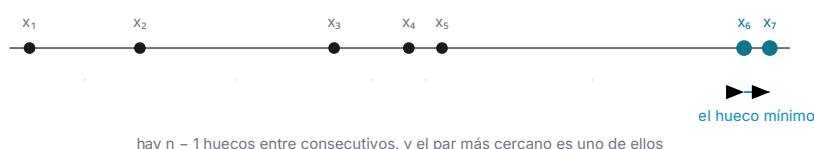


Figura 3. Ordenados, solo quedan $n - 1$ huecos que mirar.

Es la primera demostración de correctitud del curso y es corta a propósito, porque fija la forma que va a tener la larga: se supone que la solución óptima tiene cierta propiedad, se ve qué pasa si no la tiene, y se concluye.

Lo importante del calentamiento es por qué no se traslada. En la recta hay un orden total, y ese orden tiene una propiedad geométrica: si dos números están cerca, entonces están cerca en el orden. En el plano no hay ningún orden total con esa propiedad. Puedes ordenar por x , y dos puntos con el mismo x pueden estar infinitamente lejos en y . Ahí empieza la dificultad, y ahí empieza la clase.

4. Fuerza bruta

El algoritmo que siempre está disponible: mirar los $\binom{n}{2}$ pares.

```
import random
import time
from itertools import combinations

def dist2(p, q):
    return (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2

def fuerza_bruta(P):
    par = min(combinations(P, 2), key=lambda par: dist2(*par))
    return dist2(*par) ** 0.5, par

random.seed(2026)
print(f"{'n':>7} {'pares':>14} {'tiempo':>10}")
for n in [500, 1000, 2000, 4000]:
    P = [(random.random(), random.random()) for _ in range(n)]
    inicio = time.perf_counter()
    fuerza_bruta(P)
    seg = time.perf_counter() - inicio
    tiempos.setdefault("fuerza bruta", []).append((n, seg))
    print(f"{'n':>7,} {n * (n - 1) // 2:>14,} {seg:>9.3f}s")
```

n	pares	tiempo
500	124,750	0.043s
1,000	499,500	0.158s
2,000	1,999,000	0.644s
4,000	7,998,000	2.620s

Correctitud. No hay nada que demostrar: el algoritmo examina todas las soluciones candidatas y se queda con la mejor. Es la única familia de algoritmos que se justifica por agotamiento, y por eso sirve de oráculo para probar los demás.

Complejidad. $\Theta(n^2)$ comparaciones, en todos los casos, y memoria $O(1)$ además de la entrada.

Fíjate en la frase exacta: este algoritmo cuesta $\Theta(n^2)$. No hemos dicho nada sobre el problema. Esa distinción parece pedantería ahora y es el asunto central de las secciones 7 y 8.

Los tiempos de la tabla duplican n y multiplican el tiempo por cuatro, que es lo que predice el análisis. Extrapolando, un millón de puntos cuesta del orden de 10^{11} pares: días.

5. Divide y vencerás: el algoritmo

La estructura es la de siempre. Partir el problema, resolver las partes, combinar.

Ordenamos los puntos por x y cortamos por la mediana, dejando $\lceil n/2 \rceil$ puntos a la izquierda y el resto a la derecha. Llamemos L a la recta vertical que pasa por la mediana. Resolvemos recursivamente cada mitad y obtenemos δ_I y δ_D , las distancias mínimas dentro de cada una. Sea $\delta = \min(\delta_I, \delta_D)$.

Falta un caso, y es el único: el par más cercano podría tener un punto a cada lado de L . Si ese par está a distancia menor que δ , ambos puntos están a distancia menor que δ de L , porque la distancia entre los dos es al menos la diferencia de sus coordenadas x . O sea, ambos están en la **franja** de ancho 2δ centrada en L .

Eso reduce el trabajo, pero no lo suficiente. La franja puede contener los n puntos: pon todos los puntos sobre una recta vertical y δ será grande comparado con el ancho de la nube. Si comparamos todos los pares de la franja, la recurrencia queda $T(n) = 2T(n/2) + O(n^2)$, que es $O(n^2)$, y no ganamos nada.

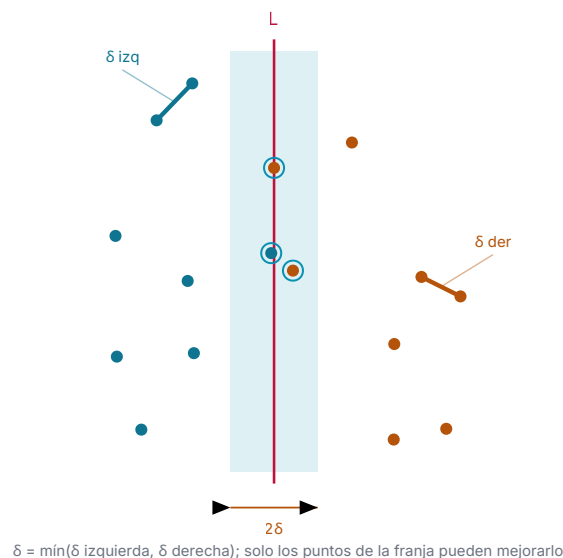


Figura 4. El estado real del algoritmo sobre 16 puntos: la mediana, los dos lados con su δ , y la franja de ancho 2δ . Solo 3 de los 16 puntos caen dentro.

Hace falta una observación más, y es el corazón de la clase.

6. Divide y vencerás: la correctitud

Lema de la franja. Ordenemos los puntos de la franja por coordenada y . Si dos puntos de la franja están a distancia menor que δ , entonces sus posiciones en ese orden difieren en a lo sumo 7.

Demostración. Sean p y q dos puntos de la franja con $d(p, q) < \delta$, y supongamos sin perder generalidad que $p_y \leq q_y$. Como $|p_y - q_y| \leq d(p, q) < \delta$, ambos están dentro del rectángulo R de ancho 2δ (el de la franja) y alto δ , con base en la altura de p , que es el de la [figura 5](#).

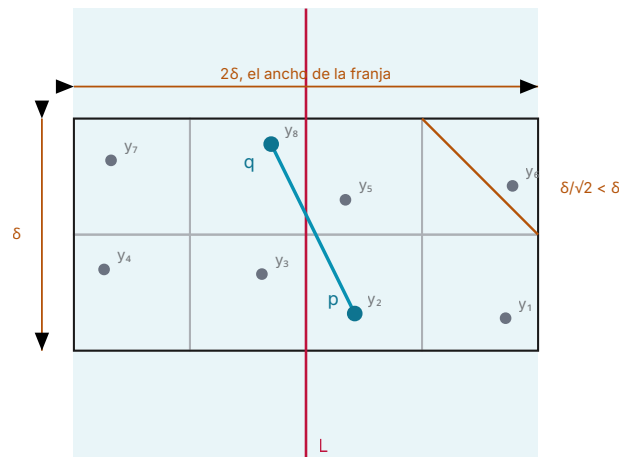
Partimos R en ocho cuadrados de lado $\delta/2$: dos filas de cuatro. El diámetro de cada cuadrado es su diagonal, $\frac{\delta}{2}\sqrt{2} = \frac{\delta}{\sqrt{2}} < \delta$.

Ahora usamos la hipótesis de inducción, y este es el paso que todo el mundo se salta. Cada cuadrado está entero de un solo lado de L , porque su lado $\delta/2$ es menor que δ , la mitad del ancho de la franja. Los puntos de un mismo lado están a distancia al menos δ entre sí, por definición de δ_I y δ_D . Como el diámetro del cuadrado es estrictamente menor que δ , **cada cuadrado contiene a lo sumo un punto de P** .

Entonces R contiene a lo sumo 8 puntos, contando p y q . Los puntos de R con altura entre p_y y q_y son consecutivos en el orden por y , así que entre p y q hay a lo sumo 6 puntos, y sus posiciones difieren en a lo sumo 7. $\square$

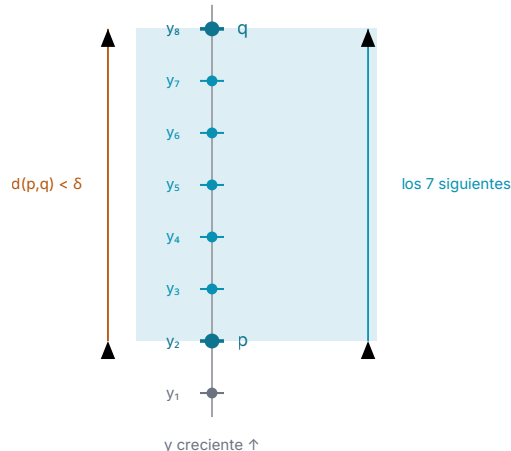
El lema dice que basta recorrer la franja en orden de y y comparar cada punto con los 7 siguientes: $O(n)$ comparaciones en total.

Correctitud del algoritmo. Por inducción fuerte sobre n . Para $n \leq 3$ la fuerza bruta es correcta. Para $n > 3$, el par más cercano está entero a la izquierda, entero a la derecha, o cruzado. Los dos primeros casos los cubre la hipótesis de inducción y dan δ . El tercero, si mejora a δ , tiene sus dos puntos en la franja y , por el lema, el algoritmo lo examina. $\square$



cada cuadrado tiene lado $\delta/2$ y por tanto a lo sumo un punto: el rectángulo entero aguanta 8

Figura 5. El rectángulo R partido en ocho cuadrados de lado $\delta/2$. Cada cuadrado está entero de un lado de L y aguanta a lo sumo un punto, así que R aguanta ocho. Los puntos van numerados por su posición en el orden de y .



q está 6 posiciones después de p : el lema garantiza que nunca pasa de 7

Figura 6. Los mismos ocho puntos, vistos solo como orden de y . Es la configuración extrema: q es el último punto que p tiene que mirar.

Esa configuración es el peor caso, no el caso típico. En una nube uniforme la franja casi nunca llega a tener ocho puntos, y el par que la mejora resulta casi siempre consecutivo en el orden de y . Por eso el 7 es una garantía y no una descripción, y por eso, como vamos a medir en la sección 7, se puede escribir 1 en lugar de 7 y pasar las pruebas.

Dos comentarios sobre el 7. Primero, es una cota, no el óptimo: con un análisis más fino del rectángulo se baja, y el ejercicio 2 pide hacerlo. Segundo, la constante no cambia la complejidad, así que en la práctica nadie se molesta; Kleinberg y Tardos usan una caja distinta y llegan a 15. Lo que importa es que exista alguna constante, porque eso es lo que convierte la franja en trabajo lineal.

7. Divide y vencerás: la complejidad

Si en cada nivel ordenamos la franja por y , cada nivel cuesta $O(n \log n)$ y la recurrencia es

$$T(n) = 2T(n/2) + O(n \log n),$$

que da $O(n \log^2 n)$. Funciona, pero se puede hacer mejor sin cambiar el algoritmo.

El arreglo es mantener los puntos ordenados por y desde el principio y repartir esa lista entre las dos mitades en tiempo lineal, igual que el paso de mezcla del ordenamiento por mezcla, pero al bajar en lugar de al subir. La recurrencia queda

$$T(n) = 2T(n/2) + O(n),$$

y por el Teorema Maestro, $T(n) = \Theta(n \log n)$. La memoria es $O(n)$.

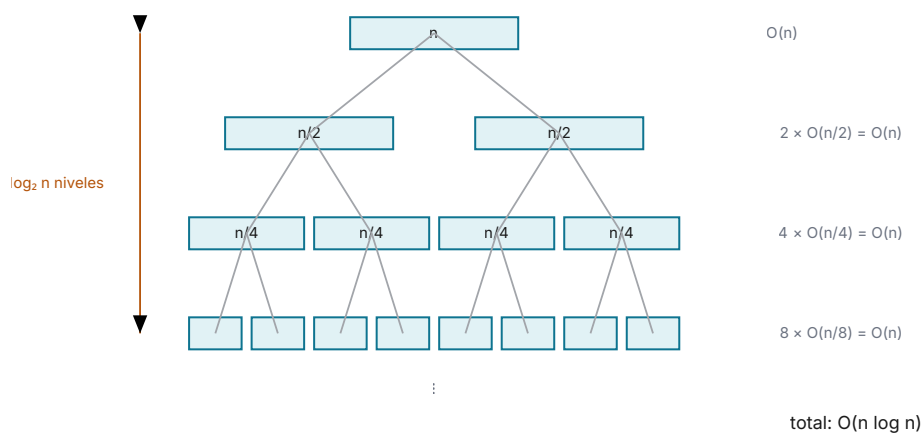


Figura 7. Repartir la lista ordenada por y cuesta $O(n)$ por nivel, y hay $\log_2 n$ niveles. Ordenar la franja en cada nivel costaría $O(n \log n)$ por nivel, y ahí sale el logaritmo de más.

Vale la pena detenerse en lo que acaba de pasar: el algoritmo es el mismo, el resultado es el mismo, y el costo bajó un factor logarítmico por una decisión de implementación. La idea y su costo son dos cosas distintas, y el análisis se hace sobre la implementación, no sobre la idea.

En el código, Q son los puntos ordenados por x , y Y es la lista de sus índices ordenada por y . La recursión trabaja sobre el intervalo $[lo, hi)$ de Q , y el reparto de Y entre las dos mitades es la comparación $j < \text{medio}$.

```
def par_mas_cercano(P, vecinos=7):
    Q = sorted(P)
    Y = sorted(range(len(Q)), key=lambda j: Q[j][1]) # por x, índices, por y
    d2, par = _rec(Q, Y, 0, len(Q), vecinos)
    return d2 ** 0.5, par

def _rec(Q, Y, lo, hi, vecinos):
    if hi - lo <= 3: # caso base: fuerza bruta
        mejor = (float("inf"), None)
        for a in range(lo, hi):
            for b in range(a + 1, hi):
                if dist2(Q[a], Q[b]) < mejor[0]:
                    mejor = (dist2(Q[a], Q[b]), (Q[a], Q[b]))
        return mejor

    medio = (lo + hi) // 2
    izq = _rec(Q, [j for j in Y if j < medio], lo, medio, vecinos)
    der = _rec(Q, [j for j in Y if j >= medio], medio, hi, vecinos)
```

```

d2, par = izq if izq[0] <= der[0] else der

delta, corte = d2 ** 0.5, Q[medio][0]
franja = [j for j in Y if abs(Q[j][0] - corte) < delta]
for a in range(len(franja)):
    for b in range(a + 1, min(a + 1 + vecinos, len(franja))):
        if dist2(Q[franja[a]], Q[franja[b]]) < d2:
            d2 = dist2(Q[franja[a]], Q[franja[b]])
            par = (Q[franja[a]], Q[franja[b]])
return d2, par

random.seed(2026)
for n in [1_000, 10_000, 100_000]:
    P = [(random.random(), random.random()) for _ in range(n)]
    inicio = time.perf_counter()
    par_mas_cercano(P)
    seg = time.perf_counter() - inicio
    tiempos.setdefault("divide y vencerás", []).append((n, seg))
    print(f"n = {n:>7,} {seg:6.3f}s")

```

```

n = 1,000      0.006s
n = 10,000     0.081s
n = 100,000    1.254s

```

Cien mil puntos en poco más de un segundo, contra los días que costaría la fuerza bruta. Verifiquemos que además da la respuesta correcta, comparando contra el oráculo de la sección 4 sobre entradas aleatorias.

```

def fallos(vecinos, instancias, n):
    malos = 0
    for _ in range(instancias):
        P = [(random.random(), random.random()) for _ in range(n)]
        if abs(fuerza_bruta(P)[0] - par_mas_cercano(P, vecinos)[0]) > 1e-12:
            malos += 1
    return malos

random.seed(1)
print(f"{'prueba':>26} {'con 7 vecinos':>15} {'con 1 vecino':>14}")
print(f"{'2000 instancias de n=12':>26} {fallos(7, 2000, 12):>15} {fallos(1, 2000, 12):>14}")
print(f"{'40 instancias de n=400':>26} {fallos(7, 40, 400):>15} {fallos(1, 40, 400):>14}")

```

	prueba	con 7 vecinos	con 1 vecino
2000 instancias de n=12		0	19
40 instancias de n=400		0	0

La columna de la izquierda es la que esperábamos: con la constante correcta no hay fallos. La de la derecha es la incómoda.

Poner 1 en lugar de 7 rompe el algoritmo, porque el lema deja de aplicarse. Pero el error aparece en cerca del 1% de las instancias pequeñas, y en las instancias grandes no aparece: con 400 puntos uniformes, el par más cercano de la franja resulta ser consecutivo en el orden por *y* prácticamente siempre, así que mirar un solo vecino acierta por casualidad. Una batería de pruebas con una docena de casos grandes —que es lo que casi todo el mundo escribe— pasa en verde sobre un algoritmo incorrecto.

La moraleja es la razón de ser de la sección 6. Probar contra entradas aleatorias verifica que implementaste lo que pensaste. No verifica que pensaste bien. Eso solo lo da la demostración, y la demostración es también lo único que te dice dónde buscar el contraejemplo: el lema usa que los puntos de un mismo lado

están a distancia $\geq \delta$, así que las entradas que lo tensan son las que amontonan puntos contra la recta de corte.

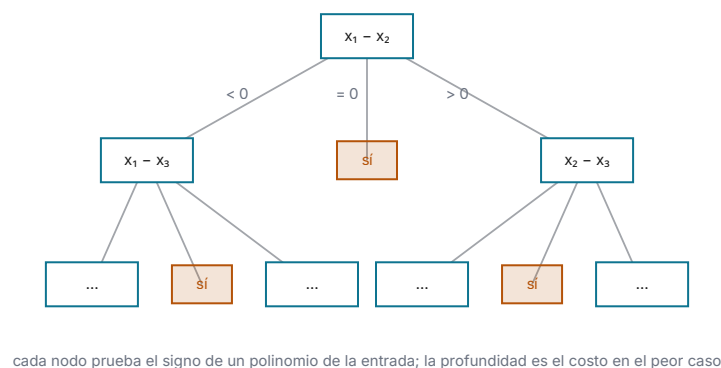
El algoritmo es de Bentley y Shamos, 1976.

8. La cota mínima

Tenemos un algoritmo $O(n \log n)$. ¿Seguimos buscando uno mejor? La quinta pregunta cambia el sujeto: en vez de preguntar cuánto cuesta este algoritmo, pregunta cuánto cuesta cualquiera.

Para demostrar algo sobre todos los algoritmos hay que decir primero qué es un algoritmo, y eso es un modelo de cómputo. El modelo estándar en geometría es el **árbol de decisión algebraico**: un árbol donde cada nodo interno evalúa el signo de un polinomio de grado acotado en las coordenadas de la entrada y ramifica según sea negativo, cero o positivo, y cada hoja da una respuesta. El costo es la profundidad del árbol, o sea el número de pruebas en el peor caso.

El modelo capta comparar, sumar, restar, multiplicar y medir distancias, que es todo lo que hacen las secciones 4 a 7. Ordenar por x es una sucesión de pruebas de signo de $x_i - x_j$; comparar distancias es una prueba de signo de $d(p, q)^2 - d(r, s)^2$, un polinomio de grado 2. El algoritmo de Bentley y Shamos es un árbol de decisión algebraico de profundidad $O(n \log n)$.



cada nodo prueba el signo de un polinomio de la entrada; la profundidad es el costo en el peor caso

Figura 8. Un árbol de decisión algebraico para distinción de elementos. Cada nodo prueba el signo de un polinomio de la entrada y ramifica en tres.

Ahora el resultado que vamos a usar, y que no demostramos.

Teorema (Ben-Or, 1983). El problema de **distinción de elementos** —dados n reales, decidir si hay dos iguales— requiere $\Omega(n \log n)$ en el modelo de árboles de decisión algebraicos.

La demostración acota el número de componentes conexas del conjunto de entradas que llevan a la respuesta "todos distintos", que son $n!$, por un teorema de Milnor y Thom sobre variedades algebraicas; un árbol de profundidad h no puede separar más de c^h componentes, y de ahí $h = \Omega(\log n!) = \Omega(n \log n)$. La citamos y seguimos.

Lo que sí demostramos es que el par más cercano hereda esa cota.

Teorema. Par más cercano requiere $\Omega(n \log n)$ en árboles de decisión algebraicos.

Demostración. Por reducción desde distinción de elementos. Dada una entrada $x_1, \dots, x_n$ de distinción de elementos, construimos la entrada de par más cercano

$$P = \{(x_1, 0), (x_2, 0), \dots, (x_n, 0)\},$$

que cuesta $O(n)$ y no usa ninguna prueba de signo. Como $d((x_i, 0), (x_j, 0)) = |x_i - x_j|$, la distancia del par más cercano de P es 0 si y solo si dos de los x_i son iguales.

Supongamos que existiera un algoritmo para par más cercano con profundidad $T(n) = o(n \log n)$. Componiéndolo con la construcción y con una prueba final de si el resultado es 0, tendríamos un algoritmo para distinción de elementos de profundidad $T(n) + 1 = o(n \log n)$, contra el teorema de Ben-Or. $\square$

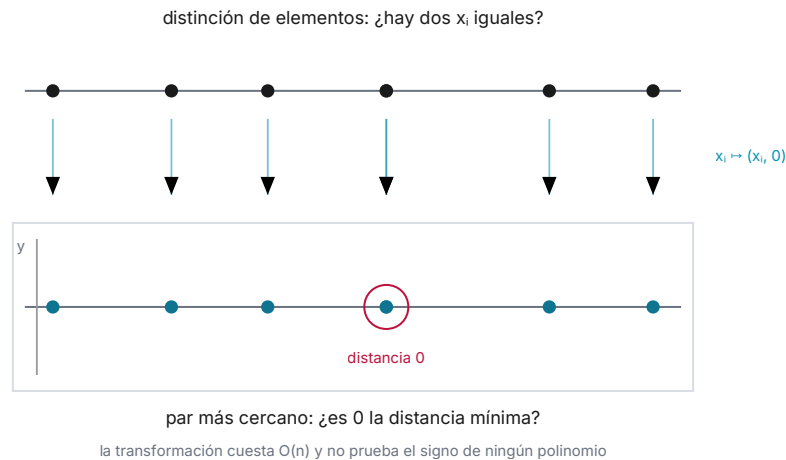


Figura 9. La reducción: los n reales se vuelven n puntos sobre el eje x . Dos valores iguales son dos puntos a distancia 0, así que la respuesta de un problema es la respuesta del otro.

Conclusión: el algoritmo de la sección 7 es óptimo, y la búsqueda se acabó.

Vale la pena mirar la forma del argumento, porque vuelve en el Tema 3 con otro objetivo. Una reducción del problema A al problema B transforma entradas de A en entradas de B de modo que la respuesta se preserve. Eso permite leerla en dos direcciones: **un algoritmo bueno para B da uno bueno para A** , que es como se usa para diseñar, y **una cota mínima para A da una cota mínima para B** , que es como la acabamos de usar. Las reducciones de la conferencia 13 en adelante son las mismas, leídas en la segunda dirección: si A es difícil y A se reduce a B , entonces B es difícil.

9. Romper la cota

Acabamos de demostrar que no se puede hacer mejor que $n \log n$. Ahora vamos a hacerlo mejor.

El algoritmo es incremental y aleatorio. Se deben las ideas a Rabin (1976); la versión que damos es la de Kleinberg y Tardos, §13.7.

Permutamos los puntos al azar: $p_1, p_2, \dots, p_n$. Los insertamos uno por uno, manteniendo en todo momento δ , la distancia del par más cercano entre los ya insertados. La estructura que sostiene el algoritmo es una **rejilla** de celdas cuadradas de lado $\delta/2$, donde la celda de un punto q es $(\lfloor q_x/(\delta/2) \rfloor, \lfloor q_y/(\delta/2) \rfloor)$, guardada en una tabla hash.

Invariante. Cada celda contiene a lo sumo un punto.

Demostración. Dos puntos en la misma celda están a distancia a lo sumo la diagonal, $\frac{\delta}{2}\sqrt{2} = \frac{\delta}{\sqrt{2}} < \delta$, y δ es la mínima distancia entre los puntos insertados. $\square$

Inserción de p_{i+1} . Cualquier punto a distancia menor que δ de p_{i+1} tiene coordenadas que difieren en menos de $\delta = 2 \cdot (\delta/2)$, o sea en menos de dos celdas en cada eje. Está entonces en el bloque de 5×5 celdas centrado en la de p_{i+1} . Consultamos esas 25 celdas, cada una con a lo sumo un punto por el invariante: 25 accesos a la tabla hash, $O(1)$.

- Si no hay ninguno más cerca que δ , entonces δ no cambia. Insertamos p_{i+1} en su celda y seguimos. Costo $O(1)$.

- Si lo hay, δ baja a un δ' nuevo, la rejilla vieja ya no sirve y hay que reconstruirla entera con lado $\delta'/2$. Costo $O(i)$.

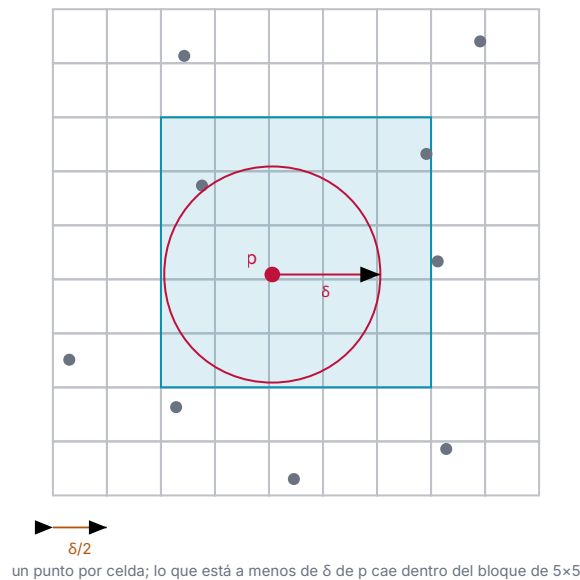


Figura 10. La rejilla real en el paso 9, recortada alrededor de p . Hay 9 celdas ocupadas de unas 121: por eso la rejilla es una tabla hash y no un arreglo.

El costo del algoritmo es entonces $O(n)$ más el costo de las reconstrucciones, y todo depende de cuántas haya.

Análisis hacia atrás. Fijemos el conjunto $S = \{p_1, \dots, p_{i+1}\}$ y olvidemos en qué orden llegó. El par más cercano de S está determinado por S : es un par, a lo sumo dos si contamos empates, y supongamos que es único. El paso $i + 1$ reconstruye si y solo si insertar p_{i+1} hizo bajar a δ , es decir, si y solo si p_{i+1} es uno de los dos puntos de ese par. Como el orden de inserción es una permutación uniforme, condicionado a cuál es el conjunto S , el elemento p_{i+1} es uniforme entre los $i + 1$ de S . Luego

$$\Pr[\text{reconstruir en el paso } i + 1] \leq \frac{2}{i + 1}.$$

El truco está en fijar el conjunto y preguntar quién fue el último, en lugar de seguir el proceso hacia adelante. De ahí el nombre.

Costo esperado. Por linealidad de la esperanza, sumando sobre los pasos,

$$\mathbb{E}[\text{costo}] = O(n) + \sum_{i=2}^{n-1} \frac{2}{i+1} \cdot O(i) = O(n) + \sum_i O(1) = O(n).$$

De paso, el número esperado de reconstrucciones es $\sum_i \frac{2}{i+1} \approx 2 \ln n$, un número tan pequeño que se puede medir.

```
import math

def par_mas_cercano_aleatorio(P, semilla=0):
    pts = list(P)
    random.Random(semilla).shuffle(pts)
    d2, par = dist2(pts[0], pts[1]), (pts[0], pts[1])
    lado = d2 ** 0.5 / 2
    celda = lambda q: (int(q[0] // lado), int(q[1] // lado))
    rejilla = {celda(q): q for q in pts[:2]}
```

```

reconstrucciones = 0

for i in range(2, len(pts)):
    p = pts[i]
    antes, (cx, cy) = d2, celda(p)
    for dx in (-2, -1, 0, 1, 2):
        for dy in (-2, -1, 0, 1, 2):
            q = rejilla.get((cx + dx, cy + dy))
            if q is not None and dist2(p, q) < d2:
                d2, par = dist2(p, q), (p, q)
    if d2 < antes:
        lado = d2 ** 0.5 / 2
        rejilla = {celda(q): q for q in pts[:i + 1]}
        reconstrucciones += 1
    else:
        rejilla[(cx, cy)] = p

return d2 ** 0.5, par, reconstrucciones

random.seed(3)
malos = sum(abs(fuerza_bruta(P)[0] - par_mas_cercano_aleatorio(P)[0]) > 1e-12
            for P in [(random.random(), random.random()) for _ in range(200)]
                    for _ in range(40))
print("fallos contra la fuerza bruta, 40 instancias de n=200:", malos, "\n")

medidas = []
print(f"{'n':>8} {'reconstrucciones':>18} {'2 ln n':>8} {'tiempo':>9}")
for n in [1_000, 10_000, 100_000]:
    P = [(random.random(), random.random()) for _ in range(n)]
    rs = [par_mas_cercano_aleatorio(P, semilla=s)[2] for s in range(3)]
    inicio = time.perf_counter()
    par_mas_cercano_aleatorio(P)
    seg = time.perf_counter() - inicio
    tiempos.setdefault("incremental aleatorio", []).append((n, seg))
    medidas.append((n, sum(rs) / len(rs), 2 * math.log(n)))
print(f"{'n':>8,} {sum(rs) / len(rs):>18.1f} {2 * math.log(n):>8.1f} {seg:>8.3f}s")

```

fallos contra la fuerza bruta, 40 instancias de n=200: 0

n	reconstrucciones	2 ln n	tiempo
1,000	13.3	13.8	0.004s
10,000	19.0	18.4	0.044s
100,000	19.0	23.0	0.706s

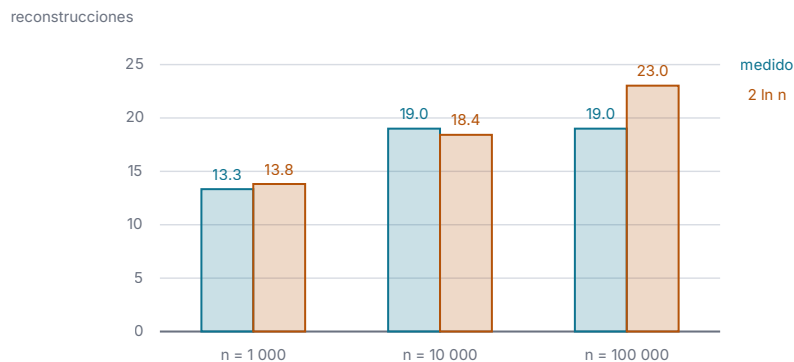


Figura 11. Lo que predice el análisis hacia atrás y lo que reconstruye el programa.

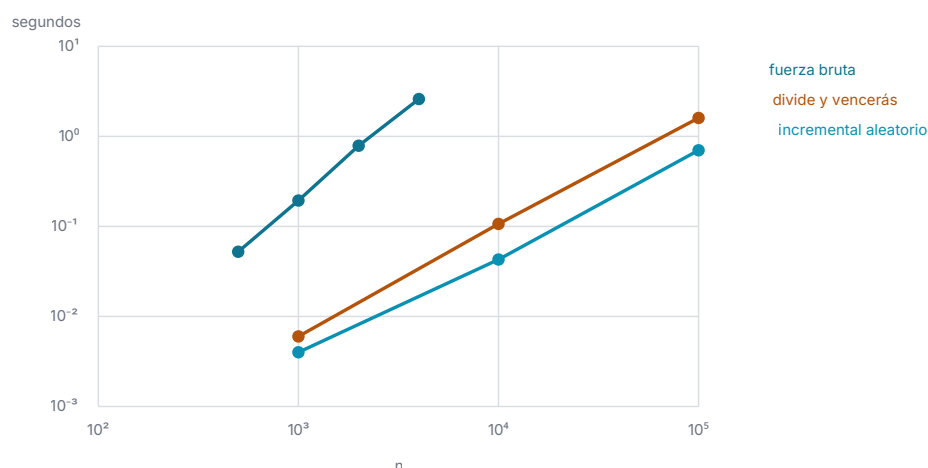


Figura 12. Los tres algoritmos de la clase, en ejes logarítmicos, con los tiempos que este documento acaba de medir. La pendiente es el exponente: 2 para la fuerza bruta, 1 para los otros dos.

Una veintena de reconstrucciones para cien mil puntos, del orden de $2 \ln n$, y la mitad del tiempo del algoritmo de la sección 7. El algoritmo lineal no es solo lineal en la pizarra.

10. Qué escondía el modelo

Tenemos una demostración de que el problema requiere $\Omega(n \log n)$ y un algoritmo que lo resuelve en $O(n)$ esperado. Una de las dos cosas tiene que estar mal, y no lo está ninguna.

Mira qué hace el algoritmo de la sección 9 que no hacía ninguno de los anteriores:

- Calcula $\lfloor q_x / (\delta/2) \rfloor$, que es lo que convierte un punto en la celda de la [figura 10](#). La función parte entera no es un polinomio, ni es continua. Un árbol de decisión algebraico no puede evaluarla.
- Usa el resultado como **dirección**: va a buscar a la tabla hash la celda cuyo índice acaba de calcular. Un árbol de decisión no tiene direccionamiento indirecto; su única operación es ramificar según el signo de un polinomio de la entrada.

La cota de Ben-Or vale para los árboles de decisión algebraicos. El algoritmo aleatorio no es uno. No hay contradicción: hay dos modelos distintos, y el segundo es estrictamente más poderoso para este problema.

Esto conecta hacia atrás con la conferencia 1. Allí la máquina de Turing, la RAM y la word RAM daban respuestas distintas a "cuánto cuesta esto", y la diferencia estaba en los exponentes. Aquí el modelo decide algo más fuerte: qué es posible. La misma pregunta, con dos modelos razonables, tiene dos respuestas separadas por un factor logarítmico.

La regla práctica que te llevas: **una cota mínima siempre viene con un modelo pegado, y la primera pregunta ante una es cuál es ese modelo y qué deja fuera**. Una cota mínima no dice "esto es imposible". Dice "esto es imposible con estas herramientas", y a veces el trabajo consiste en cambiar de herramientas.

El curso vuelve sobre las tres piezas de hoy. Las cotas mínimas y los argumentos de adversario son la conferencia 5. Los algoritmos aleatorios y el análisis de su costo esperado son la conferencia 21. Las reducciones son el Tema 3 entero, léidas al revés.

11. Resumen

- El oficio es responder cinco preguntas en orden: qué es el problema, qué algoritmo lo resuelve, por qué es correcto, cuánto cuesta y si se puede hacer mejor.
- La fuerza bruta se justifica sola y sirve de oráculo. Su costo es una cota del algoritmo, no del problema.
- Divide y vencerás resuelve el par más cercano en $O(n \log n)$. La correctitud está en el lema de la franja, que sale de un empaquetamiento y **usa la hipótesis de inducción** para acotar los puntos por cuadrado.
- La idea y su costo son cosas distintas: la misma recursión cuesta $O(n \log^2 n)$ o $O(n \log n)$ según se re-ordene la franja o se mezcle.
- Probar contra entradas aleatorias verifica que implementaste lo que pensaste, no que pensaste bien. La constante 7 del lema se puede romper sin que las pruebas se enteren.
- Par más cercano requiere $\Omega(n \log n)$ en árboles de decisión algebraicos, por reducción desde distinción de elementos. Una reducción se lee en dos direcciones: algoritmos hacia un lado, cotas mínimas hacia el otro.
- Hay un algoritmo aleatorio de costo esperado $O(n)$, por inserción incremental en una rejilla, con el análisis hacia atrás que acota en $2/i$ la probabilidad de reconstruir.
- No hay contradicción, porque ese algoritmo usa la parte entera y el direccionamiento indirecto, que el modelo de la cota no tiene. Toda cota mínima es relativa a un modelo.

Ejercicios

1. Demuestra formalmente el lema de la sección 3, cubriendo el caso de empates: si hay varios pares mínimos, ¿la demostración sigue funcionando?
2. El lema de la franja da 7. Demuestra la mejor constante que puedas con el mismo tipo de argumento, y exhibe una configuración de puntos que la alcance.
3. Par más cercano en $\mathbb{R}^3$. ¿En qué se convierte la franja? Diseña el algoritmo, demuestra su correctitud y analiza su costo. ¿Qué pasa en $\mathbb{R}^d$ con d fijo, y qué pasa si d crece con n ?
4. El par más **lejano** (el diámetro del conjunto). ¿Funciona el mismo divide y vencerás? Di exactamente cuál paso falla y por qué.
5. **Par más cercano bicromático**: cada punto es rojo o azul, y se busca el par más cercano con un punto de cada color. ¿Sobrevive el lema de la franja? Justifica.
6. Implementa el algoritmo de la sección 9, cuenta las reconstrucciones para $n = 10^3, 10^4, 10^5$ y compara con $2 \ln n$. Después quita la permutación aleatoria e inserta los puntos en el orden en que vienen: ¿qué le pasa a la cuenta?
7. Construye una entrada y un orden de inserción que fuercen al algoritmo de la sección 9 a $\Theta(n^2)$. ¿Por qué la permutación aleatoria lo hace improbable, y qué es exactamente lo que se vuelve improbable?
8. La reducción de la sección 8 va de distinción de elementos a par más cercano. ¿Hay una reducción en la dirección contraria? Si la hubiera, ¿qué se podría concluir?